Rules Engine

what business logic should and should not be configurable

A rules engine that makes the platform flexible for state agencies to self-service is a genuine competitive advantage. A rules engine that lets admins accidentally modify federally mandated calculations is a liability. The architecture already knows where that line is, it just needs to be made explicit in the admin UI's permission model.

A rules engine fits into the Configuration & Tenant Management and Forms Engine domains. The pattern is: rules stored as JSONB in cnp_global, versioned and effective-dated, evaluated at runtime by a lightweight engine. For a .NET 8 stack, Microsoft's Rules Engine (open source, takes rules as JSON, evaluates against C# models) is the natural fit.

The rules engine and admin configuration screens slot into three existing domains:

- **Forms Engine (3)** (cnp_global) — FormDefinition metadata, field labels, required/optional/hidden states, AnnualFormTypes, ReviewVersions. Admin screen = UI over these tables with version history and effective-dating.
- **Configuration & Tenant Management (2)** (cnp_global) — CnpDefaults, workflow routing rules, notification templates, alert thresholds, late claim policy, state-specific validation additions. Rules stored as JSONB with effective dates.
- **Workflow & Task Management (3)**— escalation rules, assignment rules, routing conditions. These are the most natural fit for a dedicated rules engine because they're pure business logic with no federal calculation dependency.

The LaunchDarkly integration stays for feature-level toggles (whole modules on/off). The rules engine handles the field-level and workflow-level configuration within those modules. They're complementary layers, not competing ones.

What a rules engine can legitimately manage:

✅ Types of rules a rules engine can legitimately manage:

Rule	Example

⛔ Federal calculation rules cannot go in the rules engine. Several categories of "rules" in CNP are federal statute (7 CFR 210, 225, 226) and cannot be made configurable without creating a compliance liability.

Workflow routing rules	Assign reviews to Reviewer A if risk score > 70 and program = CACFP
Escalation rules ★	If a claim hasn't been reviewed in 10 business days, send alert to supervisor
State-specific validation additions	Montana requires a food safety certificate attachment; other states don't
Form conditional logic	Show the FSMC contract section only if usesFsmc = true" (Formly expressions already do this but rules engine makes it admin-editable)
Notification triggers ★	Send email when application status changes to Pending Review
Review assignment rules ★	Auto-assign reviews based on geography or program specialization
Alert thresholds	Flag any claim where meal counts exceed 110% of prior month
State-specific late claim policy	Utah allows late originals; other states don't. This is a configurable rule, not federal law

Rule	Why it can't be in the rules engine
CEP claiming formula (ISP × 1.6)	Federal statute. If a state admin changes it, every CEP claim is wrong. USDA audit finding.
Free meal income threshold (130% poverty)	Federal statute. Non-negotiable.
60-day claim submission deadline	Federal regulation. States cannot extend it via configuration.
SFSP 2% second meal cap	Federal rule. A rules engine that lets you change it creates incorrect reimbursements.
Reimbursement rates ($4.12 free lunch)	USDA publishes these. They go in ClaimRateAmounts with EffectiveDate, not a rules engine.
Section 4/11 split formula	Federal FNS-10 reporting requirement. Cannot be modified.
Non-congregate PY2024 gate	Federal regulation.
DCH Tier I/II determination	Census-based federal methodology.

★ Implemented via `WorkflowTransitionConditions` (DaysInStatePast) and `TenantTransitionOverrides` — not a standalone rules engine entry. See Workflow Engine page.

The clean way to enforce this boundary is to label every rule with a `RuleCategory` field:

- `FEDERAL_IMMUTABLE` — stored in code, readable by admins but not editable
- `FEDERAL_CONFIGURABLE` — federal minimum with admin-configurable additions (checklist items, review questions)
- `STATE_CONFIGURABLE` — fully admin-editable, state-specific only
- `TENANT_CONFIGURABLE` — per-tenant instance of a shared template

Admin screens only surface `STATE_CONFIGURABLE` and `TENANT_CONFIGURABLE` rules as editable. Federal rules are visible for transparency but locked.